Methods for Generating Musical Melodies Using Artificial Intelligence

Ben Gangl-Lipson

December 9, 2024

Abstract

This project explores the potential for generating music using artificial intelligence, focusing specifically on melodies of different genres. Starting with a historical and contextual analysis of multiple different musical genres, we define the superlative attributes that govern an arbitrary set of rules for each genre. With that set of rules defined, a general algorithm is shown to plug those rules in and output generated melodies in a given style. Examples outputs from the program show the speed, efficiency, and accuracy that a melody can be generated with. Analysis of those outputs displays the usefulness in real world scenarios such as music composition, education, and analysis. This paper asserts that artificial intelligence possesses profound potential to shape the world of music by assisting humans in musical ideation.

1 Problem Description and Background Information

Artists around the world use artificial intelligence to help enhance their own creativity and develop their art. In music, artificial intelligence can be used to compose new works by suggesting notes, rhythms, harmonies, melodies, and more. This project focuses specifically on melodies and explores different methods for machines to generate melodies by comparing different algorithms, rules, and heuristic evaluations. Starting with a strict style of composition using the rules of counterpoint as laid out in the European classical tradition, and gradually loosening or changing the restrictions to allow for other genres of music. Comparing and contrasting results of the differences offers unique insight into how artificial intelligence suits music composition. This is a widely studied field with a vast array of approaches [2]. Machine learning can be employed to generate music based on an existing input data set [8]. Music can be categorized into a genre based on ranking of similarity from features such as pitch, chords, and rhythm similarity [5]. Many other ideas have been explored. This paper sets out to go about this problem from the angle of purely deterministic music generation via a search algorithm, like the ones used in many different artificial intelligence and planning programs.

1.1 Music Theory Background Information

This paper assumes a basic knowledge of the fundamentals of western music notation, including notes, clefs, ledger lines, rhythms, meter, time signatures, scales, key signatures, and chords. If any of these topics are unfamiliar to you, please refer to chapter 1 of the Open Music Theory educational resource linked in the bibliography. [7]

From there, we start with the simplest form of melodic composition available in the western musical tradition, species counterpoint. Specifically, we will focus on Johann Joseph Fux's Gradus ad Parnassum (Steps to Parnassus, 1725) [6], which outlines principles to write melodies under the system of species counterpoint. Fux presents these principals as steadfast and immutable rules that must be followed in order to create music that sounds good. A lot of classical music written from the 18th century onward adhered to this philosophy. In the next section, we will see how these rules can be broken in service of other styles or genres of music.

Species counterpoint begins with a single line called a canuts firmus ("fixed melody" in Latin). The cantus firmus serves as the foundation for the program to create melodic lines. These melodic lines provide the general characteristics of "smoothness", "independence", "variety", and "motion" [7]. To achieve this, the below rules are followed strictly. This paper focuses on these rules to generate single voice melodies and will not dive into the topics of first through fifth species counterpoint.

- Length of 8-16 notes.
- Arhythmic, through the use of only whole notes.
- All notes must be in the correct major key (minor is not explored in this paper).
- Begin and end on the tonic note of the key.
- Approach the tonic by stepwise motion, usually via the supertonic, sometimes from the leading tone.
- All note-to-note progressions are melodic consonances.
- Range, or interval between the lowest note and highest note in the line, of no more than a tenth, typically less than an octave.
- A single climax, or high point, that appears only once in the melody.
- Stepwise motion is preferred, but some leaps are permitted.
- Most leaps used are small.
- No repetition of "motives" or "licks".

- Any leaps of a fourth or larger must be followed by stepwise motion in the opposite direction.
- No more than two leaps in a row.
- No consecutive leaps in the same direction.
- The leading tone always progresses to the tonic.

1.2 Expanding to Other Styles & Genres of Music

Following these rules produces melodies that fit into the tradition of western classical music. To build off of that and expand into other genres we follow the same progressions that were followed throughout music history. Some genres use many of the same rules as classical, while others have different rule sets altogether. Many genres are a lot freer and more open than classical is, and therefore have far fewer rules. For example, when blues and jazz music were invented they came with their own informal guidelines. Strict rules like "All note-to-note progressions are melodic consonances" are often broken, and other rules like "No repetition of motives or licks" are ignored altogether [11]. This paper sets out to more explicitly define the rules that govern the characteristics of genres other than classical music. Each genre explored has its own dedicated program that comes with its own set of rules. To account for more open genres, many of the rules are probabilistic in nature and rely more on heuristics than on hard conditions. For all additional genres explored (swing and jazz) we need to first understand the defining attributes of melodies in that genre. Once those attributes are identified, we can then use that to create a set of unique rules that the genre follows in the same way that western classical music follows the rules of counterpoint. This allows for the use of the same general algorithms for generating the music with changed heuristic rule functions. We expose these to the programmer so that they can select which rules are disabled or enabled, and decide for themselves how strictly a given rule should be followed.

1.2.1 Swing Music

Swing music is a type of music that peaked in popularity in the United States in the early 1900s, roughly from 1915 - 1950 [10]. It served primarily as dance music with entirely new types of dance invented to suit it such as Lindy Hop, Jitterbug, and Charleston [10]. As such, swing music is known for its rhythm, phrasing, improvisation, and general forward momentum or drive. For the purposes of this study, all melodies are arhythmic, so in order to apply the characteristic swing feel and syncopation we take the melodies generated by the program and manually add rhythms. Beyond those characteristics, swing melodies are typically quite simple in nature. They usually use pentatonic notes, occasionally using the other notes of the major scale as passing tones. A key component is "memorability" or "singability". The quality of a melody in swing is often judged by how catchy the tune

is, and how easy it is to repeat or hear in your head [4]. Swing melodies often use a "call and response" form, where the same few notes are repeated with subtle variations at each repetition [4]. Finally, there are a few "special" notes in swing music that exist outside of the typical major scale. The minor 3rd and the minor 6th are used in certain scenarios to accentuate their major counterparts [4]. Their exact nature is hard to pin down, so we err on the side of being more restrictive. Following the attributes of "dancibility", "memorability", "singability", "call and response", and "special" notes will result in high quality swing melodies.

1.2.2 Jazz Music

A superset of swing music, jazz music includes many different styles. From New Orleans jazz to Kansas City jazz, to bebop, cool jazz, and hard bop, jazz includes so many different iterations that melodies are open to more variation [1]. Jazz melodies can range from fast to slow, busy to sparse, and loud to soft, but they have some characteristics which create a through line for the genre. They're generally very similar to swing melodies, though because jazz is a more open genre the melodies have more notes available to them. Rather than using mostly pentatonic notes, jazz melodies work with any note in the key and then some. One of the key elements of jazz is "freedom of expression". Elements like "singability" and "call and response" are carried over from swing music, but we can add attributes of "spontaneity" and "melodic movement" to show that more leaps are used. This supports the freedom allowed by the jazz genre [4]. The "special" notes in jazz also differ from swing. In swing we use minor 3rds and minor 6ths, but jazz opens the door to use the tritone and the minor 7th as well. The minor 3rd, tritone, and minor 7th are "blue notes" and are preferred over the minor 6th. These chromaticisms can be used similarly to how swing uses them, as passing tones, but they are occasionally moved to spontaneously [4]. Jazz, being more free, lends itself well to algorithmic generation.

2 Solution Description

To generate a melody that adheres to an arbitrary set of rules, we need an algorithm. First we define a range for our melody, with a lowest possible note and a highest possible note, then gather all of the notes in between. To do this we use a ring buffer data structure, with each note in order represented by a simple string. Natural notes are represented like "E", but notes with accidentals are represented in both sharp and flat keys like "F#/Gb". A number after each note is applied to denote the octave, so the final representation would look like "C4" or "D#/Eb3". A ring buffer is used so that the final note in the list will loop back to the start note and we can continue generating notes indefinitely until we reach the minimum and maximum bounds. An example of the API usage to generate possible notes between a given minimum and maximum is shown below in Listing 1. Note: all code for this project was written in Scala.

```
val AVAILABLE_CANTUS_FIRMUS_NOTES: List[String] =
                    GET_ALL_NOTES_BETWEEN_TWO_NOTES("E2", "E4")
// Output:
// ["E2", "F2", "F#/Gb2", "G2", "G#/Ab2", "A2", "A#/Bb2", "B2",
//   "C3", "C#/Db#3", "D3", "D#/Eb3" "E3", "F3", "F#/Gb3", "G3",
//   "G#/Ab3", "A3", "A#/Bb3", "B3", "C4", "C#/Db4", "D4", "D#/Eb4", "E4"]
```

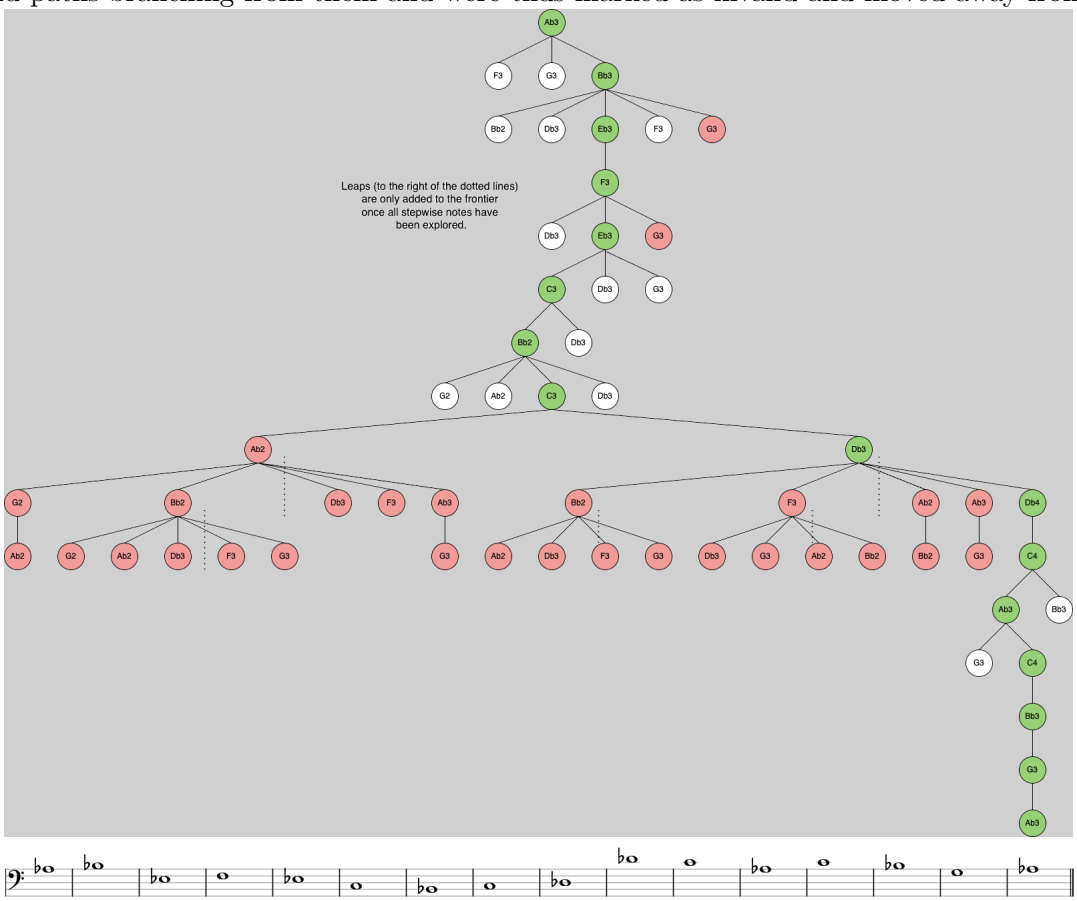
Listing 1: API usage to generate all the notes between min and max

We then randomly select a length that we want our melody to be. In counterpoint this length must be 8-16 notes, but in other genres it may be different. Next we iterate, looping through each note and searching for a good one to pick. At each iteration, we take that set of possible notes and apply rules one by one, filtering the set at each step, until we have a final list of "correct" available notes. If that list is non-empty, we randomly select a note from it, otherwise we need some way to mark that melody as invalid and to select a different melody. It's clear that a search algorithm is required in this case. The state space can easily be represented as a tree rather than a graph by treating every possible path through the valid notes as distinct. The frontier for each step through the tree is the set of distinct notes after rule filtering is applied. The branching factor will be dependent on the minimum and maximum selected notes, so it's recommended to select a range of a typical melodic voice or instrument, such as trumpet, clarinet, or saxophone. The goal condition is simply whether or not the length of the melody has been reached.

Which search algorithm suits this problem the best? Close examination of the problem shows that speed is the most important factor here, because the search space can expand to be rather large given all the possibilities. If the algorithm spends too much time examining all the options, then it becomes unusable because it can't return a valid melody in a reasonable amount of time. Additionally, cost optimality isn't required for this problem. In fact it's discouraged, because if you always return the cost optimal solution then you'll always return same melody. It's more useful to introduce randomness to more closely emulate creative human behavior. Heuristics can be used, but must also have some sort of probabilistic decision-making implemented, because they have the same problem where selecting the optimal heuristic results in the same melody generated every time.

Given all these facts about the search space, the algorithm implemented is a variant of depth limited search, with some probabilistic heuristic rule functions governing which nodes can occupy the frontier. Rules that must be followed all the time, like those found in counterpoint, simply use 100% as their probability of selection. Other rules for other genres only need to be selected some of the time. The other search algorithms considered but not ultimately selected were breadth first search, uniform cost search, and A* search. Breadth first search was found to return the same quality of results as depth limited search, but it took a much longer time to get them. Uniform cost search and A* search were both too slow, and were found to always return the same result. Depth limited search is the only algorithm explored that meets the hard requirements of speed and efficiency, as well as the soft requirements of qualitatively producing good results.

A visualization of an example tree from a run of the cantus firmus generation program has been provided on the next page, along with the cantus firmus line that was generated. Valid, or possible to traverse to, nodes are marked in white. Selected nodes are marked in green. Red marks the nodes that were initially traversed, but were found to not have any valid paths branching from them and were thus marked as invalid and moved away from.



The implementation of the algorithm easily allows for different rule functions to govern the contents of the frontier. A recursive wrapping function is used, with the goal condition set as its base case. It elegantly handles each iteration through the tree, as well invalid lines, or nodes that don't lead anywhere, by making use of Scala's Try return type. If, after all the rule filtering is applied, no nodes are found to be on the frontier, then it will hit the failure case and will backtrack up one level in the tree. If one or more valid nodes are present on the frontier, then a randomly selected node is added to the end of the list before the next iteration. The recursive wrapping function is shown in Listing 2.

```
@tailrec
private def generateMelody(
  targetLength: Int,
  tonic: String,
  initialAvailableNotes: List[String],
  melody: List[String] = List(),
  invalidLines: List[List[String]] = List(),
  invalidNotePos: Int = -1
): List[String] = {
  if (melody.length == targetLength) {
       melody // Base case is simply to check if the melody is the request length
  } else {
    generateMelodyNote(  // Otherwise randomly select a note based on our custom rule set
      targetLength,
      tonic,
      initialAvailableNotes,
      melody,
      invalidLines
    ) match {
      case Success(nextNote) =>
        // If a valid note is found, add it to the melody and iterate
        generateMelody(
          targetLength,
          tonic,
          initialAvailableNotes,
          melody :+ nextNote,
          invalidLines,
          invalidNotePos
        )
      case Failure(invalidNoteException) =>
        if (invalidNoteException.getMessage == "Can not generate.") {
          // Use a string message to handle the worst case where the rules are too strict
          melody
        } else {
          // If no valid notes are found, backtrack one level up by removing the last note
          generateMelody(
            targetLength,
            tonic,
            initialAvailableNotes,
            melody.dropRight(1),
            invalidLines :+ invalidNoteMessage    // Add the invalid line to the filter
              .filter(c => !"List()".contains(c)) // so we don't go back and forth forever
              .replace(" ", "")
              .split(",")
              .toList,
            invalidNoteMessage.last.toInt
          )
        }
    }
  }
}
```

Listing 2: Recursive wrapping function.

Within the function to generate individual notes, we do some error checking to see if we've hit the case where the rule set is so strict that a melody is unable to be generated. To do this we check to see if there have been any invalid lines found that are length 1, meaning that there are no valid nodes on the frontier from our start node and we're unable to progress anywhere. Assuming that error checking passes, we simply apply our rule set to filter the list of available nodes on the frontier. Then we send a failure if the frontier is empty and a success with a randomly selected node if the frontier is nonempty. The function to generate an individual note to be added to the melody is shown in Listing 3.

```
private def generateMelodyNote(
  targetLength: Int,
  tonic: String,
  initialAvailableNotes: List[String],
  melody: List[String],
  invalidLines: List[List[String]]
): Try[String] = {
  if (isFirstNote(melody)) {
    if (invalidLines.exists(line => line.length == 1)) {
      // If the rule set is too strict, fail with a special message.
      Failure(new Exception("Can not generate melody."))
    } else {
      // Use a special rule for getting the first note
      // Typically more restrictive, in counterpoint this must always be the tonic
      Success(getFirstNote(tonic, initialAvailableNotes)
    }
  } else {
    // For all notes past the first, apply the rule set to generate the frontier
    val frontier = applyRuleSet(targetLength, tonic, initialAvailableNotes, melody, invalidLines)
    if (frontier.isEmpty) {
      // This particular path is invalid if the frontier is empty
      Failure(new Exception(s"$melody"))
    } else {
      // Randomly select the next note
      Success(frontier(Random.nextInt(frontier.length)))
    }
  }
}
```

Listing 3: Function to generate a node (note) to be added to the path (melody).

Now that the general boilerplate around the algorithm has been explained, we can look at the interesting part, applying the rule set. It's clear that the general algorithm is agnostic to the implementation of the applyRuleSet function, which means that it's extensible. We can plug in any set of rules that we want to filter the list of available nodes and generate the frontier. The one universal rule that we must always implement is to check that the list of stored invalid lines does not contain the current list. Other than that, it's completely customizable. Within the applyRuleSet function we can continue to compose the rules into as many individual function as we'd like for clarity. A good rule of thumb to follow is to have each rule in its own function, then to aggregate those together within a filter call by chaining function calls with the && operator. An example rule is shown in Listing 4.

```
val AVAILABLE_NOTES: List[String] = GET_ALL_NOTES_BETWEEN_TWO_NOTES("C3", "B4")
val MELODIC_CONSONANCES: Set[Int] = Set(
  1, 2, 3, 4, 5, 7, 8, 9, 12
)

private def isMelodicConsonance(lastNote: String, note: String) = {
  MELODIC_CONSONANCES
    .contains(math.abs(AVAILABLE_NOTES.indexOf(lastNote) - AVAILABLE_NOTES.indexOf(note)))
}

private def applyRuleSet(
  targetLength: Int,
  tonic: String,
  initialAvailableNotes: List[String],
  melody: List[String],
  invalidLines: List[List[String]]
) = {
  initialAvailableNotes
    .filter(note => {
        !invalidLines.contains(melody :+ note) &&
        isMelodicConsonance(melody.last, note) &&
        // Other rule functions go here...
    })
}
```

Listing 4: All note-to-note progressions must be melodic consonances.

That satisfies the constraint of having deterministic rules that always result in a binary, but what about preferences? To implement a rule like "Most leaps used are small", we need some way to represent a bias without having the rule be totally strict. For this particular rule, we want the program to be biased towards using small leaps, while not completely discounting the possibility of using large leaps. Two solutions are presented. The first is to have a strict rules that states "you must only use small leaps", then create conditional logic that only applies the rule if it results in a nonempty frontier. If the resultant frontier is empty, the rule can be discarded. An example of this is shown in Listing 5. The other option is to use randomness to dictate a probability that the rule is followed. Rather than always preferring to use small leaps over large leaps, perhaps the program produces better results adhering to that preference only 80% of the time. In this case we use Scala's built in random package to create the probability, as shown in Listing 6.

```
private def applyRuleSet(
  targetLength: Int,
  tonic: String,
  initialAvailableNotes: List[String],
  melody: List[String],
  invalidLines: List[List[String]]
) = {
  val strictRulesApplied = initialAvailableNotes
    .filter(note => {
        !invalidLines.contains(melody :+ note) &&
        // Other rule functions go here...
    })

    // Note that strict preference rules must be applied after strict rules
    val smallLeapsOrStepwiseAvailable = notes.exists(note => !isALargeLeap(note, melody.last))
    if (smallLeapsOrStepwiseAvailable) {
      strictRulesApplied.filter(note => !isALargeLeap(note, melody.last))
    } else {
      strictRulesApplied
    }
}
```

Listing 5: Always prefer small leaps or stepwise motion over large leaps.

```
private def applyRuleSet(
  targetLength: Int,
  tonic: String,
  initialAvailableNotes: List[String],
  melody: List[String],
  invalidLines: List[List[String]]
) = {
  val strictRulesApplied = initialAvailableNotes
    .filter(note => {
        !invalidLines.contains(melody :+ note) &&
        (Random.between(1, 101) <= 80 && !isALargeLeap(note, melody.last)) &&
        // Other rule functions go here...
    })
}
```

Listing 6: Prefer small leaps or stepwise motion over large leaps 80% of the time.

That's everything needed to create an arbitrary set of rules for generating melodies. From here it's simply a matter of time spent experimenting and creating rules via trial-and-error. It's easy to see how, with enough time, one could generate rules to match any genre or style.

3 Experiments and Results

Results for each genre were gathered through a series of experiments to see which set of rules would best represent each genre's superlative attributes. Melodies were analyzed on a number of different instruments, as well as in real-world music composition scenarios. To fully judge the merit of generated melodies, rhythms and tempo were manually applied. The algorithm, when given a consistent set of rules none of which are too strict, is able to generate melodies extremely quickly and accurately. A test suite shows this by employing a technique called "property-based testing". For any given rule, we write a test that validates that the generated musical line follows that rule, then we run that test an arbitrary number of times, thus randomly generating a new melody each time. The more that we run the function, the higher our confidence in the accuracy of the algorithm. Additionally, adding timing metrics around the runs of functions allows us measure exactly how long it takes to generate melodies. An example test, as well as its output, has been shown in Listing 7. In this case, 1000 different melodies were generated. The algorithm is able to typically generate melodies in under a millisecond, with even the longest runs only taking a few dozen milliseconds.

```
"should ensure that all note-to-note progressions are melodic consonances" in {
  (1 to TRIES).foreach(_ => {
    melodyService.generate() match {
      case Success(melody) =>
        melody.zipWithIndex.map {
          case (note, i) =>
            if (i > 0) {
              MELODIC_CONSONANCES.contains(
                math.abs(
                  AVAILABLE_NOTES.indexOf(note) -
                    AVAILABLE_NOTES.indexOf(melody(i - 1))
                )
              ) mustBe true
            }
        }
      case Failure(e) =>
        e.printStackTrace()
        fail()
    }
  })
}

// Timing output:
// Average run time: 0.79 ms
// Longest run: 32 ms
// Total run time across 1000 runs: 790 ms
```

Listing 7: Validate the rule of melodic consonances

Next we explore each additional genre of music to see how their rule sets were created, and what exactly those rule sets are. The methodology was largely based in trial-and-error. Melodies were generated on local runs of the program, and then played on an instrument, usually piano, with different rhythms applied to determine if they the generated melodies lent themselves to human composition. Some rules were too strict and would slow down the algorithm too much, or would result in the inability to generate melodies whatsoever. Some rules were too loose and didn't affect the quality of the output at all. Eventually, a set of rules was found for a few different genres beyond cantus firmi in the western classical

tradition. These specific rules aren't perfect, but they qualitatively produce good results consistently.

3.1 Swing Music

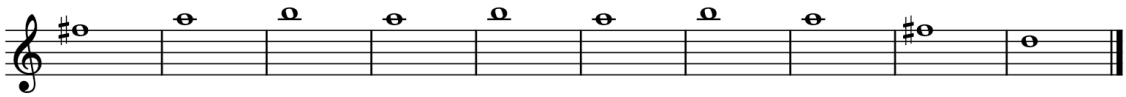
To address the defining features of swing music, the following rules were used. One addition was made to the algorithm to handle the "special" notes of minor 3rds and minor 6ths. These accentuating notes require knowledge of what notes precede them and what notes follow them, however the depth limited search is only able to look backwards, not forwards. To that end, a post-processing step was used to insert these notes at the correct positions. Once a full melody has been generated, we loop through the list and at each node look backwards and forwards to determine whether or not to add a "special" note.

- Start with the major key notes, but expand the available notes to include the minor 3rd and minor 6th via post-processing.
- Typically begin on tonic, then 5th, then 3rd, then others
- Most notes belong to the pentatonic scale of the key, others are still in the major key.
- Notes that don't belong to the pentatonic scale are typically "passing" notes that are surrounded by stepwise motion.
- Stepwise motion is preferred, but some leaps are permitted.
- Most leaps used are small.
- Encouraged repetition of "motives" or "licks" through the use of "call and response".
  - After 4-6 notes are generated, prefer to re-use previous notes in order.
- The minor 3rd is used to accentuate the major third.
  - If a major 3rd is moved to from below (from the tonic or supertonic), occasionally add a minor 3rd between them.
  - If a major 3rd is moved to from above then continues to go downwards, occasionally add a minor 3rd after the major 3rd.
  - If the melody starts with a major 3rd, occasionally add a minor 3rd before the major 3rd.
- The minor 6th is used to accentuate the major sixth.
  - If a major 6th is moved to from below, occasionally add a minor 6th before the major 6th.

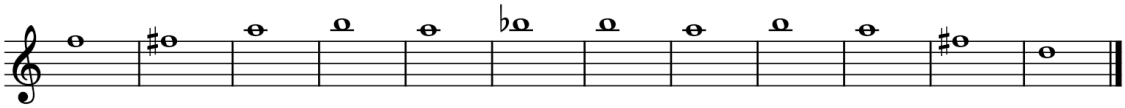
- If a major 6th is moved to from above and then continues back upwards, occasionally add a minor 6th before the major 6th.

Shown below are a few example results of melodies generated using these rules. For each melody, a first picture shows the initial generated line, a second picture shows the melody after going through the post-processing filter for "special" notes, and a third picture shows a human composed rhythm using the melody notes. This process demonstrates that the implementation of the depth limited search algorithm, in conjunction with this specific rule set, generates swing melodies with a high degree of quality and speed. We can see how the algorithm could easily be extended to generate rhythms in this manner by itself without any human intervention.

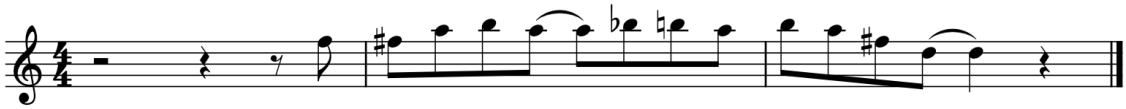
Swing melody #1 initial generated line.



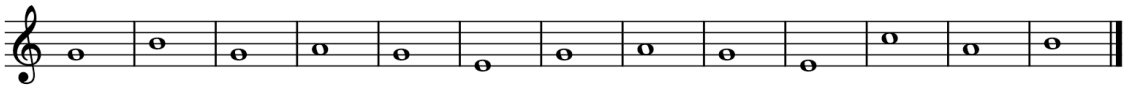
Swing melody #1 after post-processing.



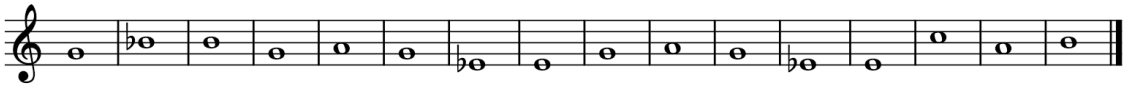
Final swing melody #1 after human composed rhythm is applied.



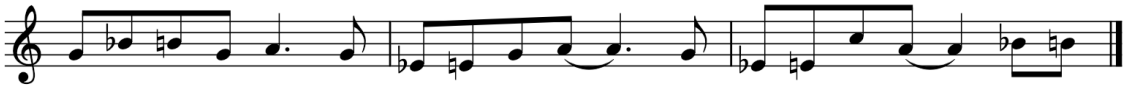
Swing melody #2 initial generated line.



Swing melody #2 after post-processing.



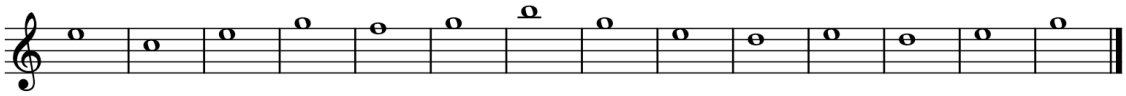
Final swing melody #2 after human composed rhythm is applied.



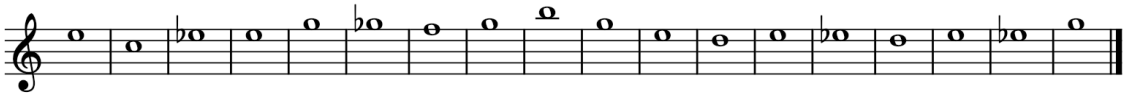
3.2 Jazz Music

Jazz music follows many of the same rules as swing music. Some pertinent alterations were made. We add the tritone and the minor 7th to the list of available notes, and make the "blue notes" more likely to be selected than other special notes in the post-processing filter. Additionally, we open up a minor probability that these notes will appear sporadically in the initial generation in addition the post-processing filter. We continue to prefer stepwise motion or small leaps, but what makes a leap "small" is changed to be larger, resulting in more spasmodic melodies. Jazz melodies often have more notes within the same time frame than their swing counterparts, since not all jazz is suited specifically for dance. This means we need to increase the length of the melody generation to accommodate this feature. Trial and error found that a length of 12-24 notes suits jazz best. Two examples are shown below of jazz melodies generated with these rule alterations in place.

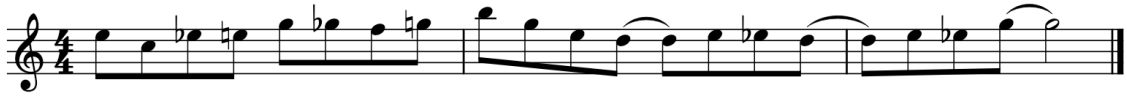
Jazz melody #1 initial generated line.



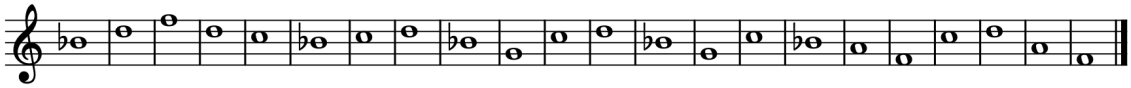
Jazz melody #1 after post-processing.



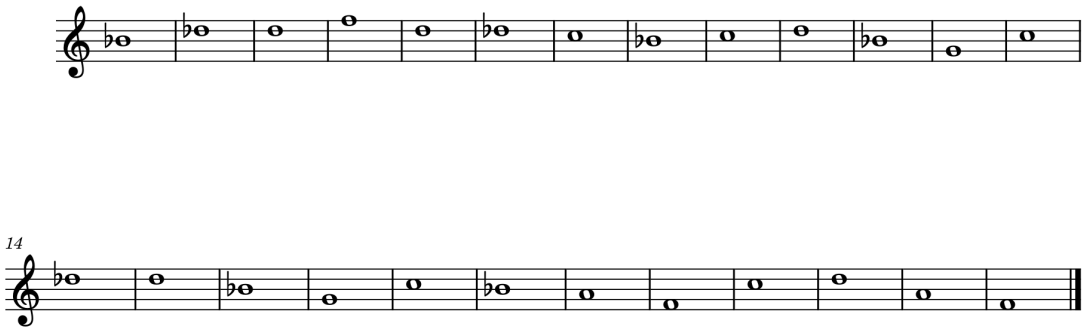
Final jazz melody #1 after human composed rhythm is applied.



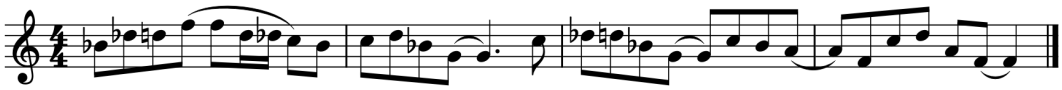
Jazz melody #2 initial generated line.



Jazz melody #2 after post-processing.



Final jazz melody #2 after human composed rhythm is applied.



4 Results Analysis

The algorithm guarantees extremely fast generation of accurate melodies. The speed at which it can generate music opens the possibility of real-time music generation, meaning music notation, composition, and processing tools could add it as a plugin to assist human composers as they compose [9]. The accuracy in the suite of tests shows that each rule set applied is guaranteed to always find a valid melody without failure. Additionally, the generated melodies qualitatively match the style and feel of each genre tested. The generated cantus firmi for classical music adhere to the metrics laid out by Johann Joseph Fux to determine a high quality cantus firmus. These qualitative metrics are "smoothness", "independence and integrity of melodic lines", "variety", "motion (toward a goal)", "pitch proximity", "step declination", "step inertia", "melodic regression", and "melodic arch" [6]. Ultimately, it comes down to a person's taste, but this paper asserts that the hundreds of generated cantus firmus analyzed adhere quite well to all of these metrics. For swing music, there are no widely agreed upon metrics so, as previously mentioned, we used the metrics of "dancibility", "memorability", "singability", "call and response", and "special" notes. While these metrics aren't all-encompassing, following them allows for the generation of some very nice swing melodies. Some notes, like the tritone and the minor 7th, which are sparingly used in swing music were entirely left out in order to maintain the guarantee of consistency. It was found that adding these notes worsened the results rather than improving them, which displays a gap in the rule set. Swing music also tends to have more drastic chord changes than western classical music, sometimes akin to altering the key itself

for only a measure or two. A song like "Keepin' Out of Mischief Now" by Fats Waller will move to a different chord, the subdominant in this case, and will treat the song as being in that key for the duration of that chord [12]. Then it moves back to the original key. This property wasn't explored thoroughly, and would require investigation into representing multi-note chords that the melody plays over rather than just the melody itself. Similarly, the jazz melodies generated follow some of the same metrics as swing, but with added metrics of "spontaneity" and "melodic movement". Jazz is the hardest genre explored to pin down because it has so much variation. It may have been better to narrow it down into other sub-genres like bebop, or hard bop. This brings up an interesting question of granularity. What exactly defines a genre of music from its counterparts, and how granular does a genre need to be in order for the melody generation algorithm to be useful? Perhaps allowing for both specific and generalized genres would be best because it'd allow the user to select their preference when they have a very specific idea in mind, while also allowing them to be guided by the algorithm when they don't. Jazz also uses chord and harmonic variations, but to an even more extreme degree than swing music. A song like "Giant Steps" by John Coltrane changes chords drastically multiple times in every measure [3]. This makes it very hard for a single-keyed melody to apply all the characteristics of jazz and necessitates an approach that takes the chord changes into account.

5 Conclusion

With this study, a general algorithm is presented to allow for an arbitrary rule set to dictate which notes are selected within a melody. This allows for artificial intelligence to assist the process of music composition in any style. Example rule sets and their corresponding results are given for 3 genres, classical, swing, and jazz. These results show the correctness of the algorithm and the usefulness of using generated notes to assist human composition. The timings provided show that the program can generate melodies extremely quickly and efficiently. There exist many real world applications for the program. It could be used as a plugin for music notation software, or in educational music software. It could also be refactored to take in a melody composed by a human and assess the quality of the melody based on the same rule sets. Future research into music generation done by artificial intelligence can go in many different directions. An analysis of chord progressions could be built into the program so that it can generate entire sections of songs rather than just a few measures. Rhythm generation could be added to give more characteristic feeling for each genre. Multiple lines and higher species counterpoint can be explored to continue the route of formal western music theory. Other methods entirely could be explored, such as machine learning. The program presented could very easily be repurposed as a means of quickly generating a set of training data for a machine learning algorithm to then continue forward with. Whatever path we take, artificial intelligence can play a critical role in the development of music composition and theory.

References

[1] Encyclopædia Britannica. Jazz. Encyclopædia Britannica, 2022. Retrieved September 2, 2022.

[2] F. Carnovalini, R. Antonio, and R. Dannenberg. Computational creativity and music generation systems: An introduction to the state of the art. *Frontiers in Artificial Intelligence*, 3:14, 2020.

[3] John Coltrane. Giant steps. Recording, 1960. From the album *Giant Steps*.

[4] Mervyn Cooke. *Jazz*. Thames and Hudson, London, 1999.

[5] J. Ens and P. Pasquier. Quantifying musical style: Ranking symbolic music based on similarity to a style. *arXiv preprint arXiv:2003.06226*, 2020.

[6] Johann Joseph Fux. *Gradus ad Parnassum (Steps to Parnassus)*. Johann Peter van Ghelen, 1725. Available in various modern editions.

[7] Mark Gotham Kyle Gullings Chelsey Hamm Bryn Hughes Brian Jarvis Megan Lavengood and John Peterson. *Open Music Theory*. Virtual Library of Virginia, 2023. Natively-online open educational resource.

[8] J. Liang. Harmonizing minds and machines: survey on transformative power of machine learning in music. *Frontiers in Neurorobotics*, 17:1267561, Nov 2023.

[9] A. Roberts and J. Engel. Magenta studio: Augmenting creativity with deep learning in ableton live. `https://research.google/pubs/magenta-studio-augmenting-creativity-with-deep-learning-in-ableton-live/`, 2019. Accessed: 2024-12-08.

[10] Howard Spring. Swing and the lindy hop: Dance, venue, media, and tradition. *American Music*, 15(2):183–207, Summer 1997.

[11] Dariusz Terefenko. *Jazz Theory From Basic to Advanced Study*. Routledge, an imprint of T&F informa, plc., 2014.

[12] Fats Waller. Keepin' out of mischief now. Recording, 1939. Recorded in 1939.